Lesson 4 Teacher's Key

Unity

INSTRUCTOR ONLY - do not distribute to students

Level 1 - Complete Worked Solution

One new method on Bullet.cs, plus a Rigidbody2D (Kinematic) and Is Trigger colliders on both prefabs:

```
// Bullet.cs - add this method:
void OnTriggerEnter2D(Collider2D other) {
    if (other.CompareTag("Enemy")) {
        Destroy(other.gameObject);
        Destroy(gameObject);
    }
}
```

Level 2 - Complete Worked Solution

Destroy() doesn't remove a GameObject instantly - it happens at the end of the current frame - so if two bullets somehow trigger on the exact same frame, both destroy calls on the Enemy are harmless; Unity simply ignores the second one, and no error is thrown. No extra code is needed for Level 2.

Level 3 - All Worked Solutions

Every stretch-goal idea from the handout, fully worked - not just one. Useful for validating each idea actually works before class, and as a ready reference no matter which one a student ends up picking.

Sound: play on hit, safely

AudioSource.PlayClipAtPoint() spawns its own temporary audio source that survives independently of the Enemy or Bullet being destroyed, so the sound always finishes playing even though both GameObjects are gone:

```
public AudioClip hitSound;

void OnTriggerEnter2D(Collider2D other) {
    if (other.CompareTag("Enemy")) {
        AudioSource.PlayClipAtPoint(hitSound, transform.position);
        Destroy(other.gameObject);
        Destroy(gameObject);
    }
}
```

Visual: give the enemy its own look

A small Enemy.cs script's Start() sets its own SpriteRenderer.color, without touching Bullet.cs at all:

```
// Enemy.cs, attached to the Enemy prefab:
void Start() {
    GetComponent<SpriteRenderer>().color = new Color(0.8f, 0.1f, 0.2f);
}
```

Structure: keep enemy-appearance logic out of Bullet.cs

OnTriggerEnter2D in Bullet.cs should only ever detect a hit and clean up (destroy the enemy, destroy itself, play the sound) - anything about how the Enemy looks on its own belongs in its own Enemy.cs. Keeping this boundary clean is what let the Visual idea above live in a totally separate script.

Complete Level 3 solution (all 3 goals combined)

Everything above combined, across Bullet.cs and Enemy.cs:

```
// Bullet.cs:
public AudioClip hitSound;

void OnTriggerEnter2D(Collider2D other) {
    if (other.CompareTag("Enemy")) {
        AudioSource.PlayClipAtPoint(hitSound, transform.position);
        Destroy(other.gameObject);
        Destroy(gameObject);
    }
}

// Enemy.cs:
void Start() {
    GetComponent<SpriteRenderer>().color = new Color(0.8f, 0.1f, 0.2f);
}
```

Alternate Path Notes: Visual Scripting

The same physics requirements apply in graphs as in code: at least one Rigidbody2D between the two colliding objects, both colliders set to Is Trigger.

The Level 3 goals map onto nodes the same way they map onto code: an Audio Source "Play Clip At Point" node for sound, a "Set Member" node on the Sprite Renderer's color for the enemy's own visual idea - all inside the Enemy prefab's own graph, kept separate from the Bullet's graph the same way Enemy.cs and Bullet.cs stay separate in the code path.

- Keeping the same Structure principle (a GameObject's own appearance/response lives with that GameObject) whether a student is in C# or graphs reinforces that it's a design idea, not a syntax rule.